Pontificia Universidad Catolica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Mecánica y Metalúrgica
ICM3818 – SNS(MF)²

Tarea 2

Parte b)

Vicente Zúñiga
31 de octubre de 2026

A partir del código en Matlab visto en clases (Clase 11 Bloque I), para la Caja de Flujo Forzada en 2D, que resulta del rotor de la ecuación de Navier-Stokes y el uso de la función de corriente (solución implementada de manera implícita en el tiempo). Considere una caja de 50 mm de largo y 20 mm de alto llena con un metal líquido justo a la temperatura de derretimiento (T_m) (si es necesario el espesor puede considerarse igual a 1 mm). Considere la densidad, conductividad térmica, capacidad calórica, viscosidad cinemática y tensión superficial del acero líquido a T_m (realice una búsqueda bibliográfica de las propiedades faltantes). El dominio debe considerar 32x32 nodos reales más los nodos fantasmas y las condiciones de borde en las 4 paredes para resolver el problema de convección en la pileta de metal fundido, como se indica esquemáticamente en la Figura 1. Considere que una fuente de calor puntual se aplica justo en la mitad superior del dominio (simulando por ejemplo un arco-eléctrico). El gradiente térmico sobre la superficie libre, hacia la izquierda y derecha, producto de esta fuente de calor, evoluciona con el tiempo y permite calcular el gradiente de tensión superficial, el cual puede ser convertido en un gradiente de velocidad (variación de la cantidad de movimiento) que varía en el tiempo y actúa justo sobre la superficie libre, tomando como dato la variación de la tensión superficial con la temperatura: $\frac{\partial \sigma_s}{\partial T_s}$.

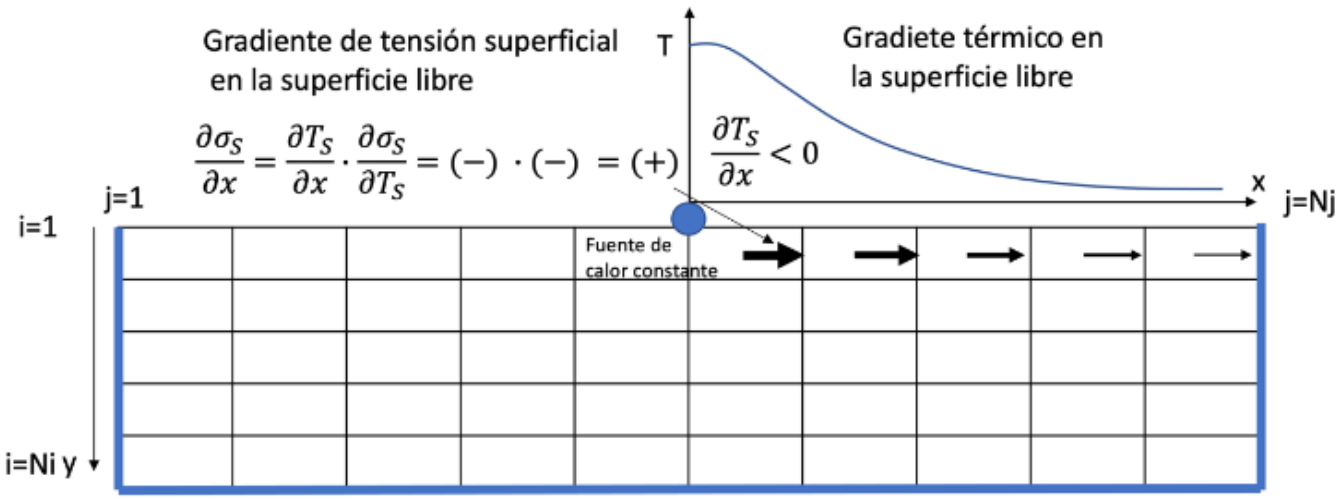


Figura 1: Bordes con velocidad $\boldsymbol{v}$ nula y vorticidad constante ω_0

De la implementación numérica del problema se pide graficar en función del tiempo (50, 100, 150 y 200 iteraciones). Para cada material obtenga:

1. Campo de temperatura en el líquido $T(x, y)$ y campo vectorial de velocidades (u, v), ambos superpuestos en el mismo gráfico (25 pts)
2. La función línea de corriente $\psi(x, y)$ (10 pts)
3. La vorticidad ω (15 pts)

Respuesta

Las propiedades de cada material a la temperatura de derretimiento T_m (para la fase líqida) se incluyen en la Tabla 1, donde se reemplazaron las propiedades entregadas de densidad, conductividad térmica y temperatura de derretimiento para mantener la consistencia con el resto de propiedades según las fuentes consultadas,

Las propiedades del acero se obtuvieron de [1]. as propiedades del Aluminio 2024-T6 corresponden a las de aluminio puro [2], y para el latón se utilizaron las propiedades del Latón C27200 a falta de datos [3], [4].

Propiedad (@T_m)	**Acero 1040**	**Aluminio 2024-T6**	**Latón C26000**
Densidad de masa ρ [kg/m^3]	7290	2391	7800
Conductividad térmica k [W/(m · K)]	34	89.3	68
Temperatura de derretimiento T_m [°C]	1472	660.32	925
Calor específico C_p [J/(kg · K)]	700.4	1127	480
Viscosidad dinámica μ [Pa · s]	$6.4 \cdot 10^{-3}$	$1.3 \cdot 10^{-3}$	$\approx 3.8 \cdot 10^{-3}$
$\partial\sigma_s/\partial T_s$ [N/(m · K)]	$-4 \cdot 10^{-4}$	$-1.3 \cdot 10^{-4}$	$\approx -3 \cdot 10^{-4}$ *

Tabla 1: Propiedades buscadas de metales a la temperatura de derretimiento T_m en fase líquida. (Nota*: $\partial\sigma_s/\partial T_s$ para el Latón corresponde al valor del cobre puro).

Ecuaciones gobernantes

El flujo incompresible en 2D se describe con la función de corriente ψ y vorticidad ω_z, como:

$$\begin{aligned} v_x &= \frac{1}{\rho}\frac{\partial \psi}{\partial y}, \quad v_y = -\frac{1}{\rho}\frac{\partial \psi}{\partial x} \\ \omega_z &= \frac{\partial v_y}{\partial x} - \frac{\partial v_x}{\partial y} = -\frac{1}{\rho}\nabla^2\psi \end{aligned} \tag{1}$$

El transporte de vorticidad, que proviene del rotor de la ecuación de Navier-Stokes, queda como:

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + (\boldsymbol{v} \cdot \boldsymbol{\nabla})\boldsymbol{\omega} = \frac{\mu}{\rho}\nabla^2\boldsymbol{\omega}$$

$$\frac{\partial \omega_z}{\partial t} + v_x\frac{\partial \omega_z}{\partial x} + v_y\frac{\partial \omega_z}{\partial y} = \frac{\mu}{\rho}\nabla^2\omega_z$$

$$\rho\frac{\partial \omega_z}{\partial t} + \frac{\partial \psi}{\partial y}\frac{\partial \omega_z}{\partial x} - \frac{\partial \psi}{\partial x}\frac{\partial \omega_z}{\partial y} = \mu\nabla^2\omega_z \tag{2}$$

La ecuación de calor, que viene del balance de energía considerando una fuente, queda como:

$$\rho C_p\left(\frac{\partial T}{\partial t} + (\boldsymbol{v} \cdot \boldsymbol{\nabla})T\right) = k\nabla^2 T + \dot{q}_c + \mu\underbrace{2\dot{\varepsilon}_{xy}\left(\frac{\partial v_x}{\partial y} + \frac{\partial v_y}{\partial x}\right)}_{\approx 0}$$

$$\rho C_p\left(\frac{\partial T}{\partial t} + v_x\frac{\partial T}{\partial x} + v_y\frac{\partial T}{\partial y}\right) = k\nabla^2 T + \dot{q}_c$$

$$\rho C_p\left(\frac{\partial T}{\partial t} + \frac{1}{\rho}\frac{\partial \psi}{\partial y}\frac{\partial T}{\partial x} - \frac{1}{\rho}\frac{\partial \psi}{\partial x}\frac{\partial T}{\partial y}\right) = k\nabla^2 T + \dot{q}_c \tag{3}$$

Condiciones de borde

En las paredes sólidas (borde inferior y laterales), hay velocidad nula, vorticidad constante y flujo de calor nulo:

$$v_x = \left.\frac{\partial \psi}{\partial y}\right|_{x=0, x=L_x, y=0} = v_y = -\left.\frac{\partial \psi}{\partial x}\right|_{x=0, x=L_x, y=0} = 0$$

$$w_z\mid_{x=0} = -\frac{2\psi\mid_{x=0^+}}{\Delta x^2}, \quad w_z\mid_{x=L_x} = -\frac{2\psi\mid_{x=L_x^-}}{\Delta x^2}, \quad w_z\mid_{y=0} = -\frac{2\psi\mid_{y=0^+}}{\Delta y^2} \tag{4}$$

$$\left.\frac{\partial T}{\partial x}\right|_{x=0, x=L_x} = \left.\frac{\partial T}{\partial y}\right|_{y=0} = 0$$

En la superficie libre (borde superior), hay un esfuerzo de corte tangencial igual al gradiente de tensión superficial, que depende de la temperatura, y no hay velocidad vertical ni flujo de calor:

$$w_z\mid_{y=L_y} = -\left.\frac{\partial v_x}{\partial y}\right|_{y=L_y} = -\frac{1}{\mu}\frac{\partial \sigma_s}{\partial T_s}\left.\frac{\partial T}{\partial x}\right|_{y=L_y} \tag{5}$$

$$v_y|_{y=L_y} = -\left.\frac{\partial \psi}{\partial x}\right|_{y=L_y} = 0, \quad \left.\frac{\partial T}{\partial y}\right|_{y=L_y} = 0$$

Discretización numérica

Se usan $N_x = N_y = 32$ nodos reales más los nodos fantasmas en la frontera. El paso de tiempo es $\Delta t = 0.005$ s y se realizan 200 iteraciones para la fuente de calor $\dot{q} = 200$ W. Tanto para la discretización de (1), (2) como para (3) se emplea un esquema implícito centrado en espacio.

Función de corriente (1):

$$w_{i,j}^t = -\frac{\psi_{i+1,j}^t - 2\psi_{i,j}^t + \psi_{i-1,j}^t}{\rho \Delta x^2} - \frac{\psi_{i,j+1}^t - 2\psi_{i,j}^t + \psi_{i,j-1}^t}{\rho \Delta y^2}$$

Vorticidad (2):

$$\rho \frac{\omega_{i,j}^{t+1} - \omega_{i,j}^t}{\Delta t} + \frac{\psi_{i,j+1}^{t+1} - \psi_{i,j-1}^{t+1}}{2\Delta y} \frac{\omega_{i+1,j}^{t+1} - \omega_{i-1,j}^{t+1}}{2\Delta x} - \frac{\psi_{i+1,j}^{t+1} - \psi_{i-1,j}^{t+1}}{2\Delta x} \frac{\omega_{i,j+1}^{t+1} - \omega_{i,j-1}^{t+1}}{2\Delta y} =$$
$$\frac{\mu}{\Delta x^2}\left(\omega_{i+1,j}^{t+1} - 2\omega_{i,j}^{t+1} + \omega_{i-1,j}^{t+1}\right) + \frac{\mu}{\Delta y^2}\left(\omega_{i,j+1}^{t+1} - 2\omega_{i,j}^{t+1} + \omega_{i,j-1}^{t+1}\right)$$

Temperatura (3):

$$\rho C_p \left(\frac{T_{i,j}^{t+1} - T_{i,j}^t}{\Delta t} + \frac{\psi_{i,j+1}^{t+1} - \psi_{i,j-1}^{t+1}}{\rho 2\Delta y} \frac{T_{i+1,j}^{t+1} - T_{i-1,j}^{t+1}}{2\Delta x} - \frac{\psi_{i+1,j}^{t+1} - \psi_{i-1,j}^{t+1}}{\rho 2\Delta x} \frac{T_{i,j+1}^{t+1} - T_{i,j-1}^{t+1}}{2\Delta y} \right) =$$
$$\frac{k}{\Delta x^2}\left(T_{i+1,j}^{t+1} - 2T_{i,j}^{t+1} + T_{i-1,j}^{t+1}\right) + \frac{k}{\Delta y^2}\left(T_{i,j+1}^{t+1} - 2T_{i,j}^{t+1} + T_{i,j-1}^{t+1}\right) + \dot{q}_{i,j}$$

Resultados

De los mapas de temperatura superpuestos la velocidad (Figuras 2, 3 y 4) se logra observar el desarrollo de corrientes. El fluido se desplaza desde el centro del dominio hacia las paredes laterales que están más frías. Como el coeficiente térmico de tensión superficial ($\partial \sigma_s / \partial T_s$) es negativo para los tres casos, la tensión superficial aumenta en las zonas de menor temperatura y esto hace que se arrastre el fluido hacia los extremos laterales de la superficie libre. Por otro lado, en las Figuras 8, 9 y 10 se aprecia que la vorticidad ω_z se concentra principalmente en la superficie libre, que es donde existe el mayor gradiente de temperaturas. La vorticidad es simétrica con respecto al eje vertical donde se aplica la fuente de calor, pero con valores opuestos que indican el sentido de giro del fluido. Ocurre lo mismo para la función de corriente ψ en las Figuras 5, 6 y 7, que es simétrica con respecto al eje vertical y tiene valores opuestos a ambos lados del eje.

1. Campos de temperatura y velocidad

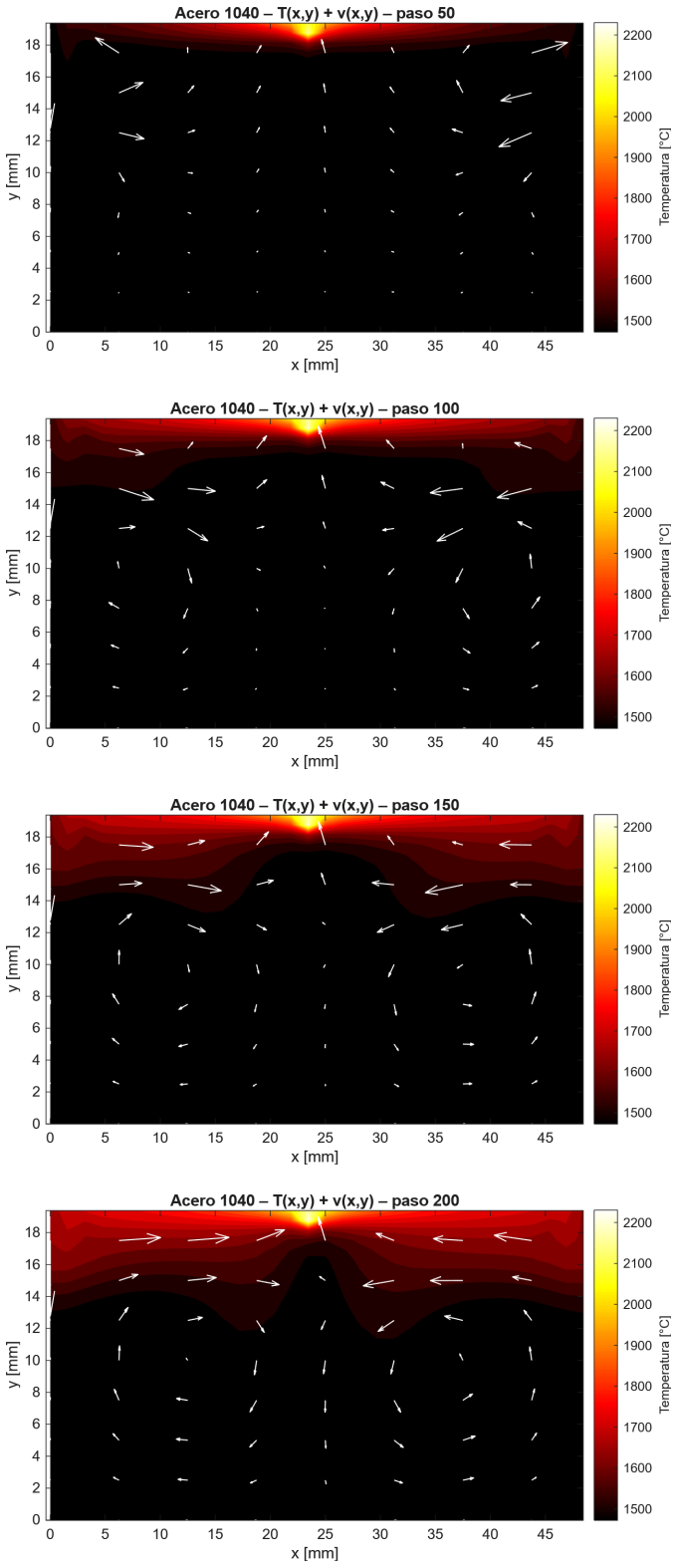


Figura 2: Temperatura T y velocidad $\boldsymbol{v}$ para el Acero 1040 en las iteraciones [50, 100, 150 y 200].

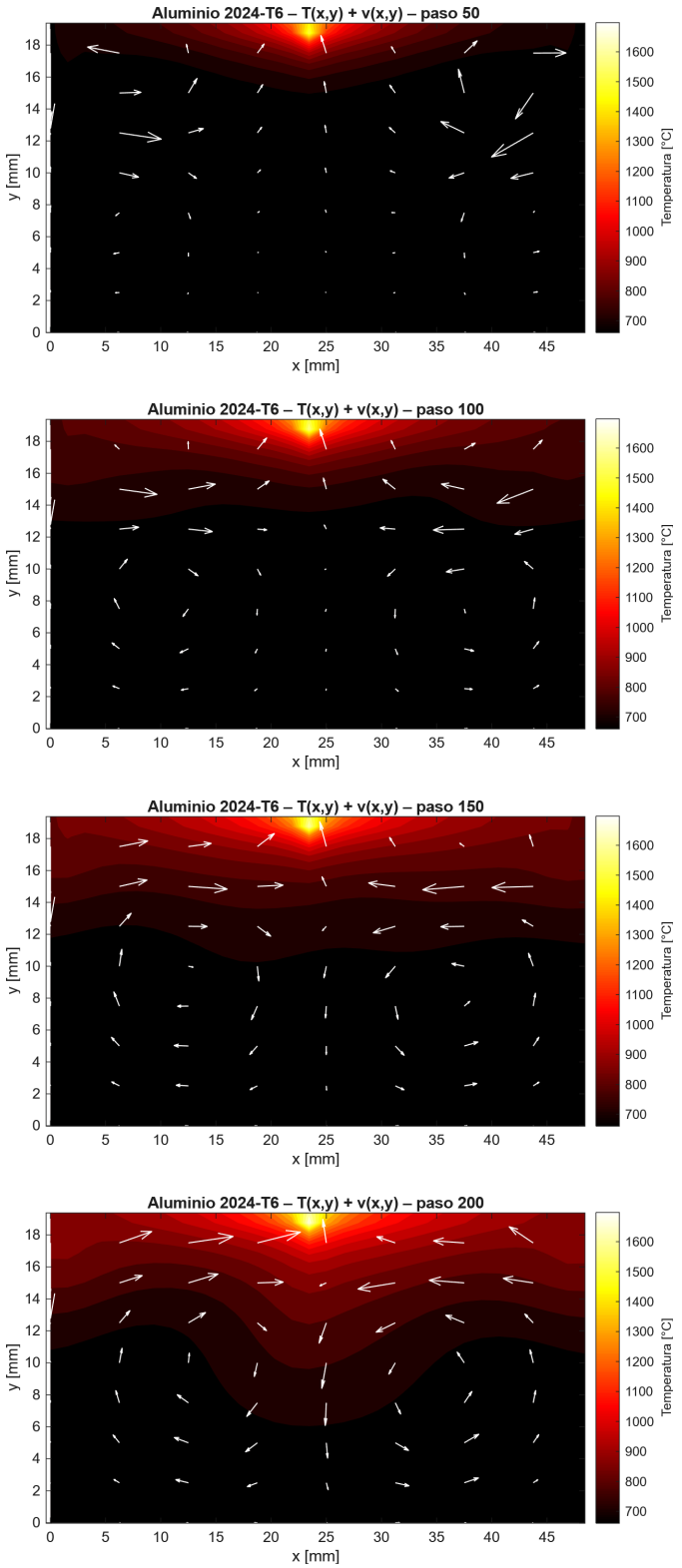


Figura 3: Temperatura T y velocidad $\boldsymbol{v}$ para el Aluminio 2024 en las iteraciones [50, 100, 150 y 200].

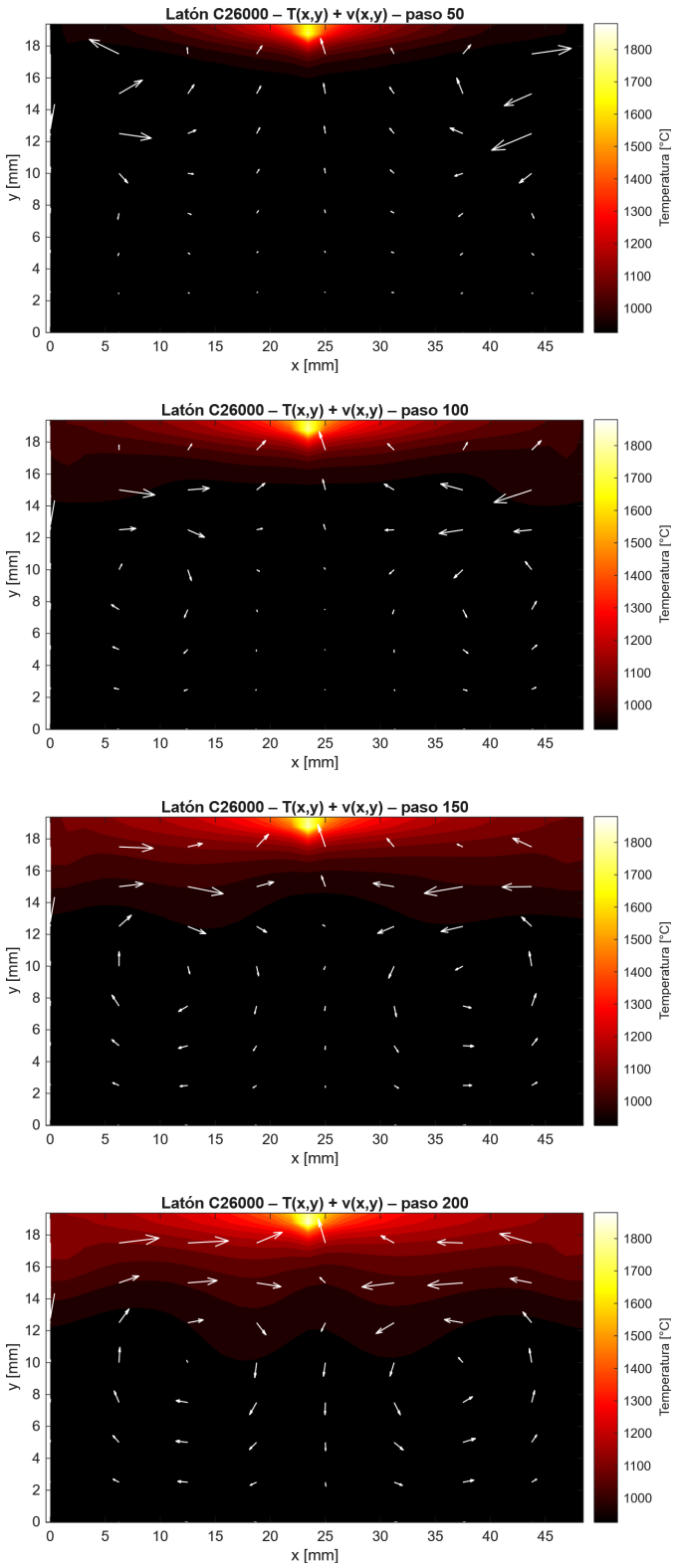


Figura 4: Temperatura T y velocidad $\boldsymbol{v}$ para el Latón C26000 en las iteraciones [50, 100, 150 y 200].

2. Función de corriente

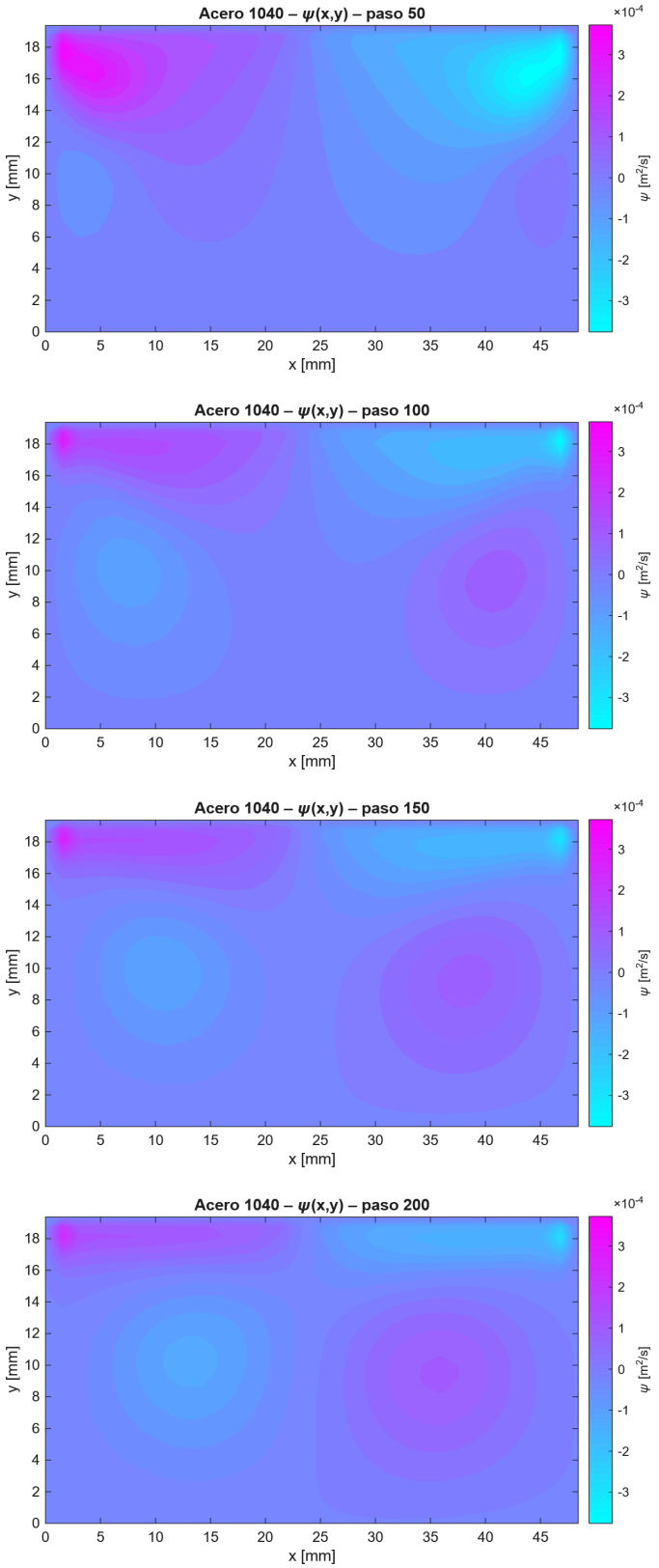


Figura 5: Función de corriente ψ para el Acero 1040 en las iteraciones [50, 100, 150 y 200].

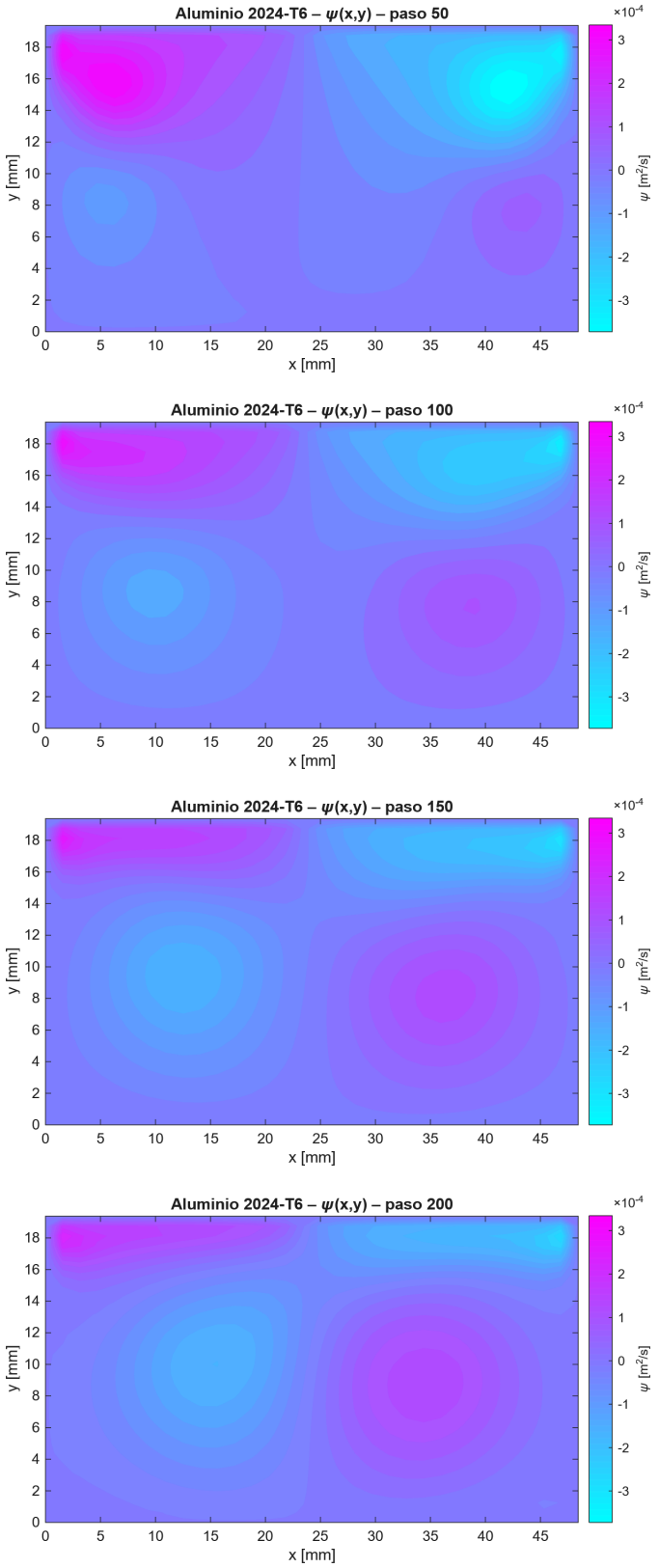


Figura 6: Función de corriente ψ para el Aluminio 2024-T6 en las iteraciones [50, 100, 150 y 200].

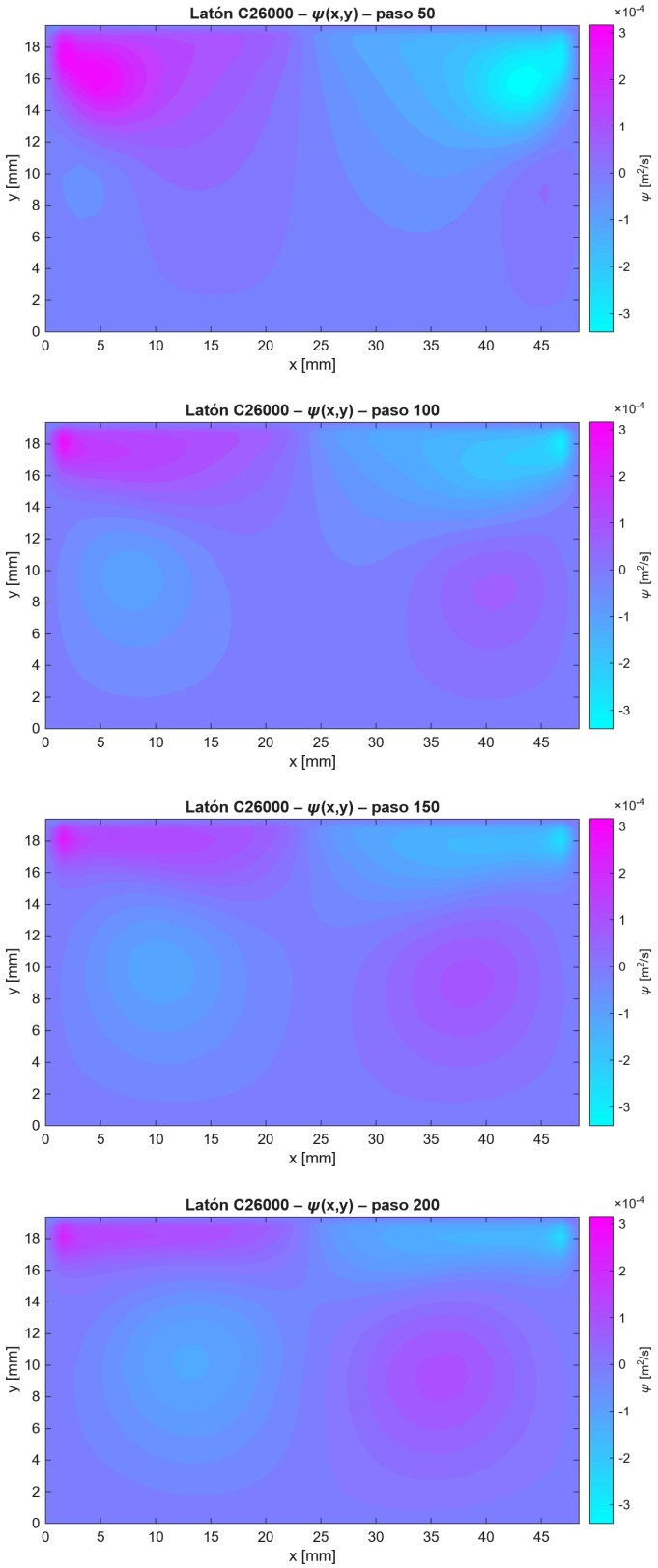


Figura 7: Función de corriente ψ para el Latón C26000 en las iteraciones [50, 100, 150 y 200].

3. Vorticidad

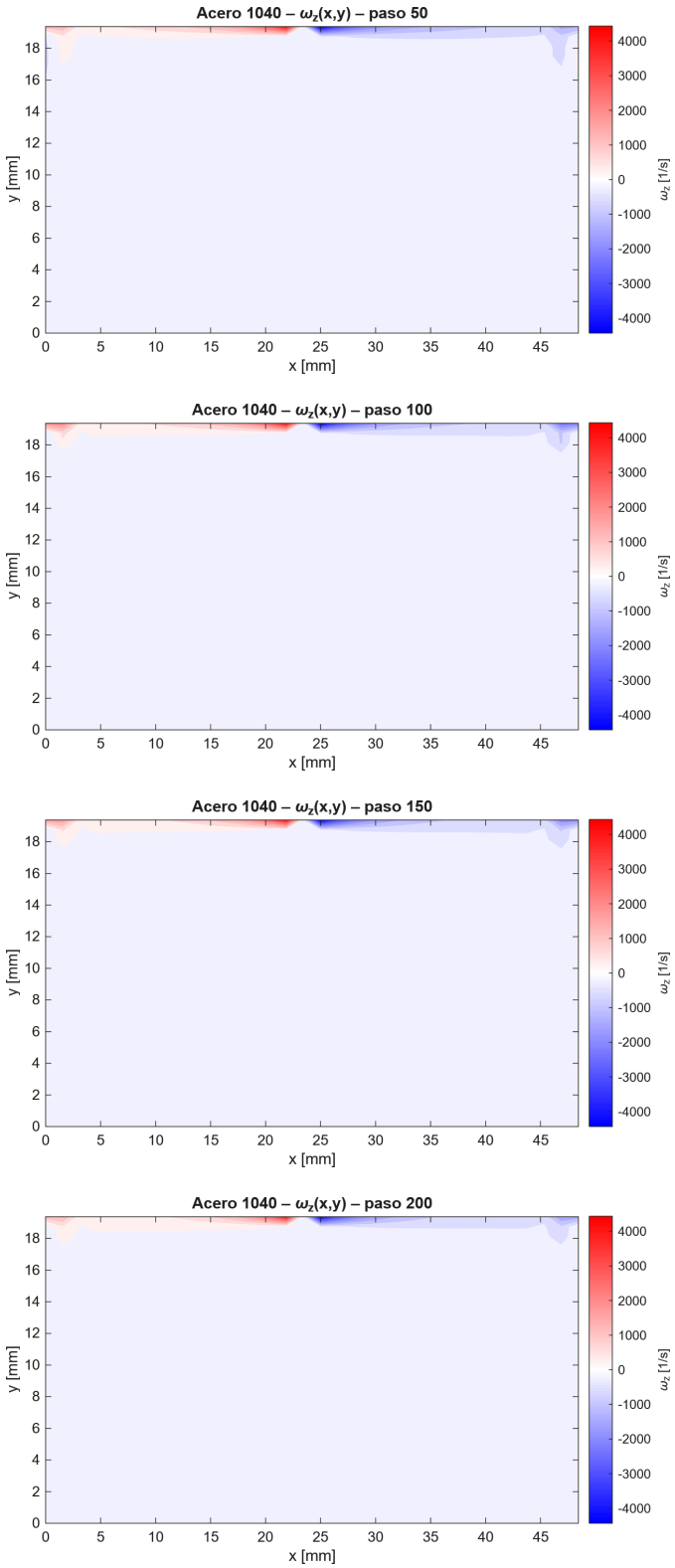


Figura 8: Vorticidad ω_z para el Acero 1040 en las iteraciones [50, 100, 150 y 200].

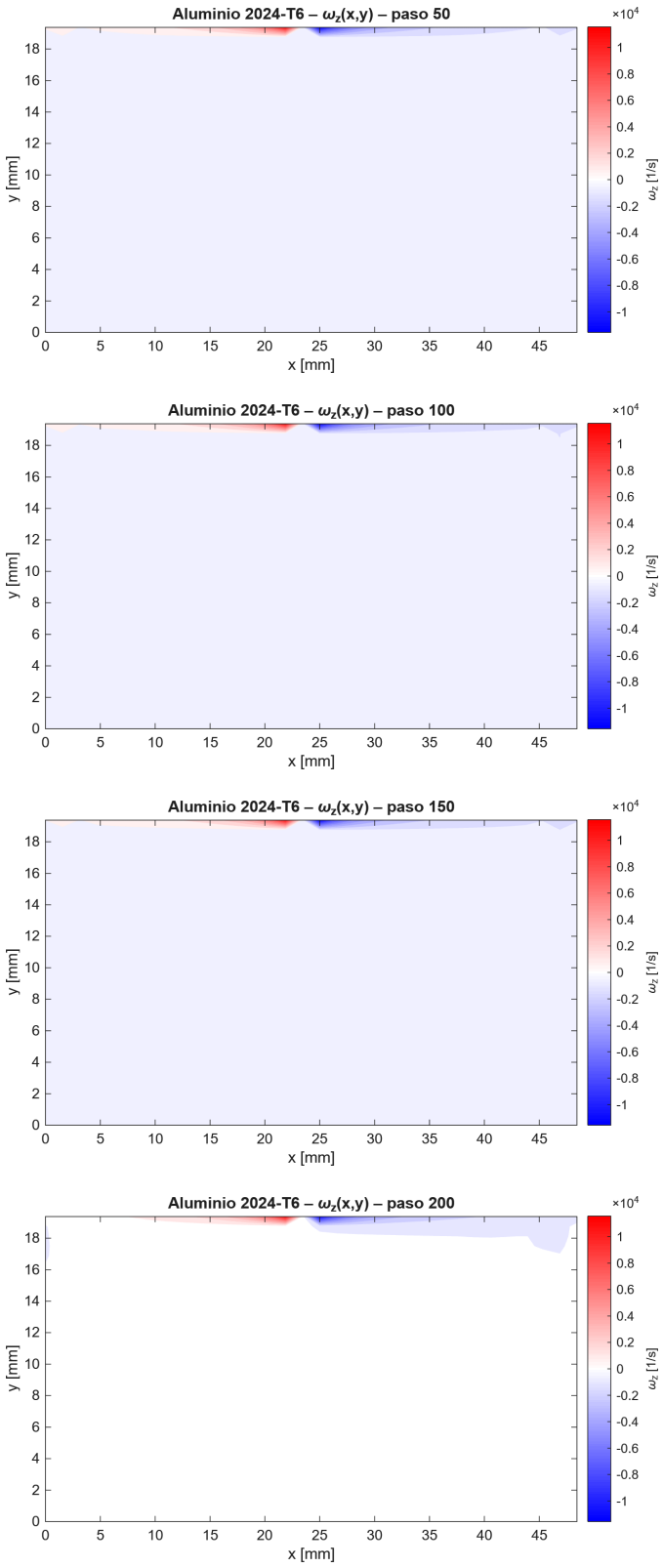


Figura 9: Vorticidad ω_z para el Aluminio 2024-T6 en las iteraciones [50, 100, 150 y 200].

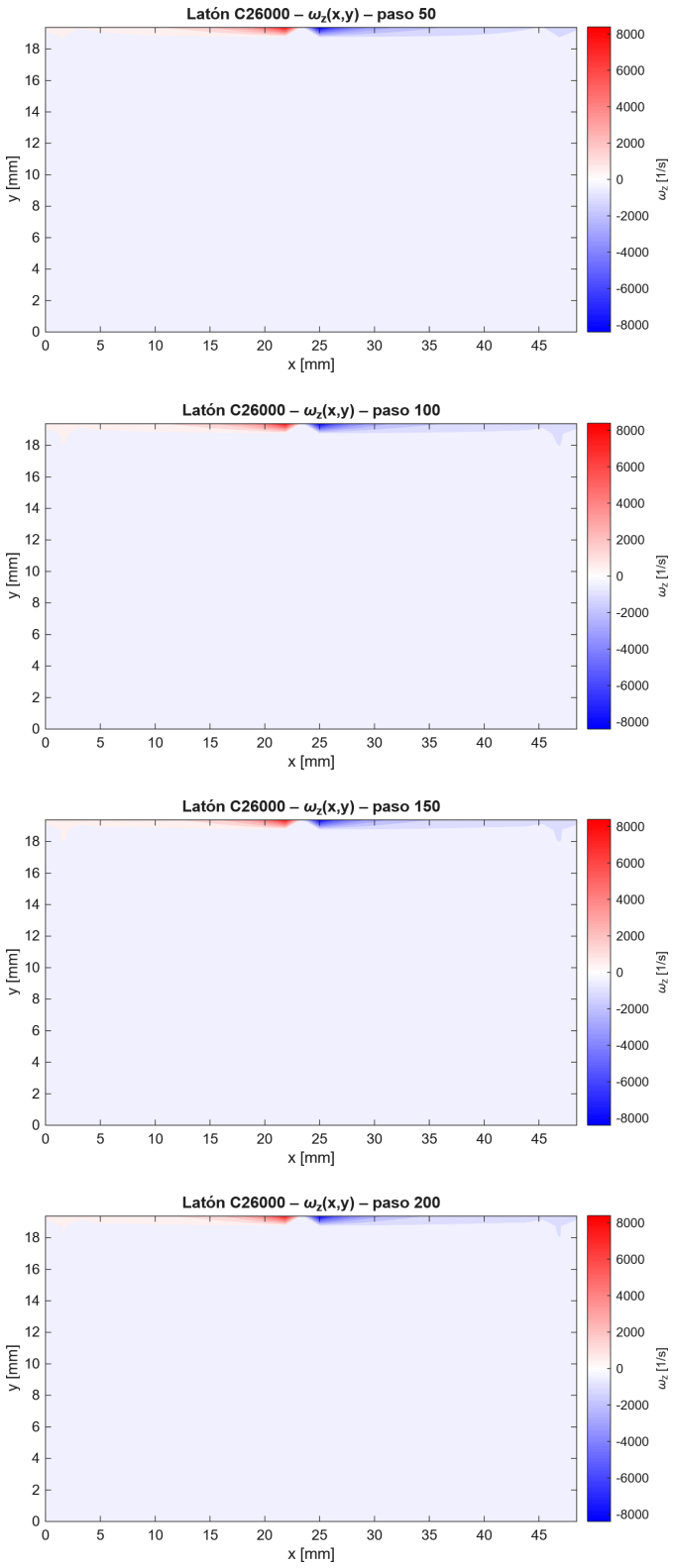


Figura 10: Vorticidad ω_z para el Latón C26000 en las iteraciones [50, 100, 150 y 200].

Bibliografía

[1] W. Ou, T. Mukherjee, G. L. Knapp, Y. Wei, y T. DebRoy, «Fusion zone geometries, cooling rates and solidification parameters during wire arc additive manufacturing», diciembre de 2018.

[2] Matthias Leitner, Thomas Leitner, Alexander Schmon, Kirmanj Aziz, y Gernot Pottlacher, «Thermophysical Properties of Liquid Aluminum», junio de 2017.

[3] V. Glück Nardi, T. Greß, B. Tonn, H. Schmidt, y W. Volk, «Fast reactive interdiffusion between solid brass and liquid aluminium», diciembre de 2023.

[4] T. Greß *et al.*, «Vertical continuous compound casting of copper aluminum bilayer rods», febrero de 2021.

Anexo

Código Matlab entregado (Clase 11 Bloque 1)

```
Nx = 20;
Ny = 20;
MaxStep = 100;
visc = 0.1;
dt = 0.005;
h = 1.0 /Nx;
Uo = 2.0;
MaxIt = 100;
MaxErr = 1e-3;

% Inicializa todas las variables en 0
fc = zeros(Nx, Ny);
w = zeros(Nx, Ny);
wo = zeros(Nx, Ny);
x = zeros(Nx, Ny);
y = zeros(Nx, Ny);

% Genera matriz - x e y - con coo de los nodos en espacio
for i = 1:Nx
 for j = 1:Ny
   x(i, j) = h * (j-1);
   y(i, j) = h * (i-1);
 end
end

wo = fc; % Guarda la funcion de corriente calculada para compararla
for isetp = 1:MaxStep
 for iter = 1:MaxIt
   for i = 2:Ny-1
     for j = 2: Nx-1
      fc(i, j) = 0.25 * (fc(i+1,j) + fc(i-1,j) + fc(i,j+1) + fc(i,j-1) + (h*h)*w(i,j))
     end
   end

   Err = 0.0;
   for i = 1:Ny
     for j = 1:Nx
       Err = Err + abs(wo(i,j) - fc(i,j));
     end
   end
   if Err <= MaxErr
    break
   end

 % Condiciones de borde para vorticidad en fronteras
 w(1,2:Nx-1) = -2.0 * fc(2,2:Nx-1)/(h*h) + Uo/(h*h); % pared superior
 w(Ny,2:Nx-1) = -2.0 * fc(Ny-1,2:Nx-1)/(h*h); % pared inferior
 w(2:Ny-1,1) = -2.0 * fc(2:Ny-1,2)/(h*h); % pared derecha
 w(2:Ny-1,Nx) = -2.0 * fc(2:Ny-1,Nx-1)/(h*h); % pared izquierda
 wo = w;

 for i = 2:Ny-1
  for j = 2:Nx-1
   w(i,j) = wo(i,j) + dt*(0.25 * ( (fc(i+1,j)-fc(i-1,j)) * (wo(i,j+1)-wo(i,j-1))
- ...
     (fc(i,j-1)-fc(i,j+1)) * (wo(i+1,j)-wo(i-1,j)) )/(h*h) + ...
       4*visc  * (wo(i+1,j)+wo(i-1,j)+wo(i,j+1)+wo(i,j-1)  -4.0*wo(i,j))  /
(h*h) );
   end
 end
end
```

Código MATLAB

El código completo de la simulación se divide en un archivo main.m que resuelve las ecuaciones discretizadas sobre el dominio, y el archivo guardar_fotos.m que genera y guarda los gráficos.

Código main.m:

```
% Params
Nx = 32;
Ny = 32;
Lx = 50e-3;
Ly = 20e-3;
dz = 1e-3;

dx = Lx / Nx;
dy = Ly / Ny;

Nxt = Nx + 2;
Nyt = Ny + 2;

dt       = 0.005;
MaxStep  = 200;
Times    = [50 100 150 200];

MaxIt_T  = 5000;
MaxIt_w  = 5000;
MaxIt_psi = 5000;
tol_T    = 1e-8;
tol_w    = 1e-8;
tol_psi  = 1e-8;

mat_names = {'Acero 1040', 'Aluminio 2024-T6', 'Latón C26000'};
K        = [34, 89.3, 68];
Cp       = [700.4, 1127, 480];
Rho_liq  = [7290, 2391, 7800];
Tm_C     = [1472, 660.32, 925];
Tm_K     = Tm_C + 273.15;
mu_dyn   = [6.4e-3, 1.3e-3, 3.8e-3];
nu       = mu_dyn ./ Rho_liq;
dsigma_dT = [-4.0e-4, -1.3e-4, -3.0e-4];
Alpha    = K ./ Rho_liq ./ Cp;

Q    = 200;
qvol = Q / (dx * dy * dz);
i_src = Nyt - 2;
j_src = round(Nx/2) + 1;
denom_psi = 2/dx^2 + 2/dy^2;

for m = 1:3
   fprintf('\n=== Material %d: %s ===\n', m, mat_names{m});

   k_th  = K(m);
   cp_m  = Cp(m);
   rho   = Rho_liq(m);
   nu_m  = nu(m);
   alpha = Alpha(m);
   dsdT  = dsigma_dT(m);
   mu_m  = mu_dyn(m);
   Tm    = Tm_K(m);

   psi = zeros(Nyt, Nxt);
   w   = zeros(Nyt, Nxt);
   T   = Tm * ones(Nyt, Nxt);

   T(1,:) = T(2,:);  T(Nyt,:) = T(Nyt-1,:);
   T(:,1) = T(:,2);  T(:,Nxt) = T(:,Nxt-1);

   psi_save = zeros(Nyt, Nxt, numel(Times));
   w_save   = zeros(Nyt, Nxt, numel(Times));
   T_save   = zeros(Nyt, Nxt, numel(Times));
   u_save   = zeros(Nyt, Nxt, numel(Times));
   v_save   = zeros(Nyt, Nxt, numel(Times));

   for istep = 1:MaxStep
      vx = zeros(Nyt, Nxt);   % u = dpsi/dy
      vy = zeros(Nyt, Nxt);   % v = -dpsi/dx

      for i = 2:Nyt-1
         for j = 2:Nxt-1
            vx(i,j) =  (psi(i+1,j) - psi(i-1,j)) / (2*dy);
            vy(i,j) = -(psi(i,j+1) - psi(i,j-1)) / (2*dx);
         end
      end


      T_old = T;
      for iter = 1:MaxIt_T
         T_prev = T;

         T(1,:)   = T(2,:);    T(Nyt,:) = T(Nyt-1,:);
         T(:,1)   = T(:,2);    T(:,Nxt) = T(:,Nxt-1);

         for i = 2:Nyt-1
            for j = 2:Nxt-1
               u_p = max( vx(i,j), 0) / dx;
               u_m = max(-vx(i,j), 0) / dx;
               v_p = max( vy(i,j), 0) / dy;
               v_m = max(-vy(i,j), 0) / dy;

               diag_T = 1/dt + u_p + u_m + v_p + v_m + 2*alpha/dx^2 +
2*alpha/dy^2;

               rhs_T = T_old(i,j)/dt;
               if (i == i_src) && (j == j_src)
                  rhs_T = rhs_T + qvol/(rho*cp_m);
               end

               T(i,j) = ( rhs_T ...
                  + (alpha/dy^2 + v_p) * T(i-1,j) ...
                  + (alpha/dy^2 + v_m) * T(i+1,j) ...
                  + (alpha/dx^2 + u_p) * T(i,j-1) ...
                  + (alpha/dx^2 + u_m) * T(i,j+1) ...
                  ) / diag_T;
            end
         end

         T(1,:)   = T(2,:);    T(Nyt,:) = T(Nyt-1,:);
         T(:,1)   = T(:,2);    T(:,Nxt) = T(:,Nxt-1);
```

```
        if max(abs(T(2:Nyt-1,2:Nxt-1) - T_prev(2:Nyt-1,2:Nxt-1)), [],
'all') <= tol_T
            break
        end
    end



    w(2, 2:Nxt-1)      = -2.0 * psi(3, 2:Nxt-1)      / dy^2;
    w(2:Nyt-1, 2)      = -2.0 * psi(2:Nyt-1, 3)      / dx^2;
    w(2:Nyt-1, Nxt-1)  = -2.0 * psi(2:Nyt-1, Nxt-2)  / dx^2;

    for j = 2:Nxt-1
        dTdx_surf = (T(Nyt-1, j+1) - T(Nyt-1, j-1)) / (2*dx);
        w(Nyt-1, j) = -(dsdT/mu_m) * dTdx_surf;
    end

    w_old = w;
    for iter = 1:MaxIt_w
        w_prev = w;
        for i = 3:Nyt-2
            for j = 3:Nxt-2
                u_p = max( vx(i,j), 0) / dx;
                u_m = max(-vx(i,j), 0) / dx;
                v_p = max( vy(i,j), 0) / dy;
                v_m = max(-vy(i,j), 0) / dy;

                diag_w = 1/dt + u_p + u_m + v_p + v_m + 2*nu_m/dx^2
+ 2*nu_m/dy^2;

                rhs_w = w_old(i,j)/dt;
                w(i,j) = ( rhs_w ...
                    + (nu_m/dy^2 + v_p) * w(i-1,j) ...
                    + (nu_m/dy^2 + v_m) * w(i+1,j) ...
                    + (nu_m/dx^2 + u_p) * w(i,j-1) ...
                    + (nu_m/dx^2 + u_m) * w(i,j+1) ...
                    ) / diag_w;
            end
        end
        if max(abs(w(3:Nyt-2,3:Nxt-2) - w_prev(3:Nyt-2,3:Nxt-2)), [],
'all') <= tol_w
            break
        end
    end

    omega  = 1.5;
    for iter = 1:MaxIt_psi
        psi_prev = psi;
        for i = 3:Nyt-2
            for j = 3:Nxt-2
                psi_GS = ( (psi(i+1,j) + psi(i-1,j))/dy^2 ...
                        + (psi(i,j+1) + psi(i,j-1))/dx^2 ...
                        + w(i,j) ) / denom_psi;
                psi(i,j) = (1 - omega)*psi(i,j) + omega*psi_GS;
            end
        end
        psi(2,:) = 0;
        psi(Nyt-1,:) = 0;
        psi(:,2) = 0;
        psi(:,Nxt-1) = 0;

        if max(abs(psi(3:Nyt-2,3:Nxt-2) - psi_prev(3:Nyt-2,3:Nxt-2)), [],
'all') <= tol_psi
            break
        end
    end

    if any(istep == Times)
        idx = find(Times == istep);
        psi_save(:,:,idx) = psi;
        w_save(:,:,idx)   = w;
        T_save(:,:,idx)   = T;
        u_save(:,:,idx)   = vx;
        v_save(:,:,idx)   = vy;
        Tmax = max(T(2:Nyt-1,2:Nxt-1),[],'all') - 273.15;
        wmax = max(abs(w(:)));
        psimax = max(abs(psi(:)));
        fprintf('  Paso %3d | T_max = %7.1f C | |w|_max = %.2e | |psi|
_max = %.2e\n', ...
            istep, Tmax, wmax, psimax);
    end
end

fname = sprintf('resultados_m%d.mat', m);
save(fname, 'psi_save','w_save','T_save','u_save','v_save', ...
'Times','dx','dy','Nx','Ny','Nxt','Nyt','Tm_C','m','mat_names','Tm_K');
end

fprintf('\nSimulacion completa.\n');
```

Código guardar_fotos.m:

```
mkdir('fotos');
for m = 1:3
    fname = sprintf('resultados_m%d.mat', m);
    load(fname);
    Tm_label = Tm_C(m);
    mname    = mat_names{m};

    x_mm = (0:Nx-1) * dx * 1e3;
    y_mm = (0:Ny-1) * dy * 1e3;

    T_mat   = T_save(2:Nyt-1, 2:Nxt-1, :) - 273.15;
    psi_mat = psi_save(2:Nyt-1, 2:Nxt-1, :);
    w_mat   = w_save(2:Nyt-1, 2:Nxt-1, :);

    T_lims   = [min(T_mat(:)), max(T_mat(:))];
    psi_lims = [min(psi_mat(:)), max(psi_mat(:))];

    w_max_abs = max(abs(w_mat(:)));
    if w_max_abs == 0, w_max_abs = 1; end
    w_lims = [-w_max_abs, w_max_abs];

    if T_lims(1) == T_lims(2), T_lims(2) = T_lims(1) + 1; end
    if psi_lims(1) == psi_lims(2), psi_lims(2) = psi_lims(1) + 1e-6; end

    for ti = 1:numel(Times)
        t_step = Times(ti);

        T_plot   = T_mat(:, :, ti);
        psi_plot = psi_mat(:, :, ti);
        w_plot   = w_mat(:, :, ti);
        u_plot   = u_save(2:Nyt-1, 2:Nxt-1, ti);
        v_plot   = v_save(2:Nyt-1, 2:Nxt-1, ti);

        fig1 = figure('Visible','off','Color','w','Position',[0 0 700 350]);
        ax1  = axes('Parent', fig1);

        contourf(ax1, x_mm, y_mm, T_plot, 20, 'LineColor', 'none');
        colormap(ax1, hot);
        cb1 = colorbar(ax1);
        cb1.Label.String = 'Temperatura [°C]';
        clim(ax1, T_lims);
        hold(ax1, 'on');

        skip = 4;
        [Xq, Yq] = meshgrid(x_mm(1:skip:end), y_mm(1:skip:end));
        Uq = u_plot(1:skip:end, 1:skip:end);
        Vq = v_plot(1:skip:end, 1:skip:end);
        quiver(ax1, Xq, Yq, Uq, Vq, 'w', 'AutoScaleFactor', 0.8, ...
            'LineWidth', 0.8, 'MaxHeadSize', 0.5);
        xlabel(ax1, 'x [mm]'); ylabel(ax1, 'y [mm]');
        title(ax1, sprintf('%s – T(x,y) + v(x,y) – paso %d', mname, t_step));
        hold(ax1, 'off');
        exportgraphics(fig1, sprintf('fotos/T_vel_m%d_t%d.png', m, t_step), 'Resolution', 200);
        close(fig1);

        fig2 = figure('Visible','off','Color','w','Position',[0 0 700 350]);
        ax2  = axes('Parent', fig2);

        contourf(ax2, x_mm, y_mm, psi_plot, 20, 'LineColor', 'none');
        colormap(ax2, cool);
        cb2 = colorbar(ax2);
        cb2.Label.String = '\psi [m^2/s]';
        clim(ax2, psi_lims);
        xlabel(ax2, 'x [mm]'); ylabel(ax2, 'y [mm]');
        title(ax2, sprintf('%s – \\psi(x,y) – paso %d', mname, t_step));
        exportgraphics(fig2, sprintf('fotos/psi_m%d_t%d.png', m, t_step), 'Resolution', 200);
        close(fig2);

        fig3 = figure('Visible','off','Color','w','Position',[0 0 700 350]);
        ax3  = axes('Parent', fig3);

        contourf(ax3, x_mm, y_mm, w_plot, 20, 'LineColor', 'none');
        colormap(ax3, bluewhitered_approx(256));
        cb3 = colorbar(ax3);
        cb3.Label.String = '\omega_z [1/s]';
        clim(ax3, w_lims);
        xlabel(ax3, 'x [mm]'); ylabel(ax3, 'y [mm]');
        title(ax3, sprintf('%s – \\omega_z(x,y) – paso %d', mname, t_step));
        exportgraphics(fig3, sprintf('fotos/omega_m%d_t%d.png', m, t_step), 'Resolution', 200);
        close(fig3);
    end
end
fprintf('Listo/\n');

function cmap = bluewhitered_approx(n)
    half = floor(n/2);
    blue_to_white = [linspace(0,1,half)', linspace(0,1,half)', ones(half,1)];
    white_to_red  = [ones(n-half,1), linspace(1,0,n-half)', linspace(1,0,n-half)'];
    cmap = [blue_to_white; white_to_red];
end
```